1.3 Selektion (Auswahl von Objekten)

Bei der Selektion im WHERE-Befehl können Objekte (Zeilen) einer Klasse mithilfe von Bedingungen ausgewählt werden.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Bedingung>
```

BEISPIEL · ALLE USER AUS KIEL

```
SELECT *
FROM users
WHERE city = 'Kiel'
```

Vergleichsoperationen – In SQL wird die Gleichheit mit „=“ und nicht mit „==“ überprüft. Texte stehen in einfachen Anführungszeichen `'Kiel'`, Zahlen ohne. Sonstige Operationen sind analog zu Java.

Gleich	Ungleich	Größer	Kleiner	Größergleich	Kleinergleich
`=`	`!=`	`>`	`<`	`>=`	`<=`

Ähnlichkeit (LIKE) – Mit dem Schlüsselwort LIKE kann man Objekte auswählen, deren Attribute einer vorgegebenen Zeichenkette ähnlich sind. Das Zeichen `_` dient als Platzhalter für ein beliebiges Zeichen und `%` als Platzhalter für eine Zeichenkette beliebiger Länge. Die Bedingung `username LIKE 'levi%'` erfüllen beispielsweise Objekte mit dem Username „levi.kicks“ oder „levi_fan_09“, jedoch nicht „mama_von_levi“. LIKE unterscheidet dabei nicht zwischen Groß- und Kleinschreibung.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Attribut> LIKE <Muster>
```

BEISPIEL · ALLE USER, DEREN USERNAME MIT „LEVI“ BEGINNT

```
SELECT username, name
FROM users
WHERE username LIKE 'levi%'
```

Logische Operationen (AND, OR, NOT) – Auch die logischen Operationen gibt es in SQL. Diese können den **WHERE**-Befehl erweitern.

ALLE USER, DIE AUS BERLIN KOMMEN UND FELIX HEISSEN

```
SELECT username, name, city
FROM users
WHERE city = 'Berlin' AND name LIKE 'Felix%'
```

ALLE USER, DIE AUS BERLIN KOMMEN ODER FELIX HEISSEN

```
SELECT username, name, city
FROM users
WHERE city = 'Berlin' OR name LIKE 'Felix%'
```

ALLE USER, DIE AUS BERLIN KOMMEN UND NICHT FELIX HEISSEN

```
SELECT username, name, city
FROM users
WHERE city = 'Berlin' AND name NOT LIKE 'Felix%'
```

Hinweis: `AND name NOT LIKE` lässt sich in SQL auch mit `AND NOT name LIKE` schreiben.

Leere Werte (IS NULL) – Hat ein User etwas nicht angegeben, steht dort **NULL** („nichts“). NULL ist kein Wert: `city = NULL` findet deshalb nie etwas. Stattdessen schreibt man `city IS NULL` bzw. `city IS NOT NULL`.

Wechselwirkungen von Einschränkungen – AND bindet stärker als OR (wie Punkt- vor Strichrechnung). Um Bedingungen logisch zu kapseln, können – wie in der Mathematik – Klammern genutzt werden.

Alle User aus Kiel, egal ob männlich oder weiblich – nur eine Möglichkeit funktioniert

MÖGLICHKEIT 1

```
SELECT name, city, gender
FROM users
WHERE city = 'Kiel' AND
      gender = 'male' OR
      gender = 'female'
```

MÖGLICHKEIT 2

```
SELECT name, city, gender
FROM users
WHERE city = 'Kiel' AND
      (gender = 'male' OR
       gender = 'female')
```

Aufgabe 3

Notiere für die folgenden Abfragen (ab Aufgabe b) einen SQL-Befehl.

a) Teste beide Möglichkeiten vom letzten Abschnitt und finde heraus, worin der Unterschied der Ausgaben liegt. Welche Möglichkeit passt zur Aufgabe?

b) Gib alle User aus, die nicht aus Deutschland kommen. Schau dir dein Ergebnis genau an: Stimmt es wirklich?

c) Zeige nur Levi Schumacher an. Wie viele Treffer bekommst du? Wie kannst du sicherstellen, dass du genau einen bestimmten Levi bekommst?

d) Prüfe, ob es minderjährige User gibt.

e) ★ Finde zuerst heraus, wann levi.kicks geboren ist. Gib dann alle User aus, die im selben Jahr geboren sind, und ordne sie nach Geburtsdatum.

f) Erstelle eine Liste mit Usern, deren E-Mail-Adresse zu deinem Netzwerk gehört (sie endet auf *@name-deines-netzwerks.test*).

g) Gib eine Liste aller männlichen User an, die größer als 1,85 m sind. Ordne die Liste nach Wohnort.

h) Prüfe, ob es einen User gibt, der Felix heißt und nicht aus Berlin kommt.

i) ★ Die Bundeswehr sucht Rekrutinnen und Rekruten. Gib die Namen und das Alter aller Männer über 165 cm und aller Frauen über 160 cm aus (es werden keine Panzerfahrerinnen und Panzerfahrer gesucht).

j) Erna Lüders sucht ihren Enkel. Sein Vorname war entweder Lasse oder Lars. Er kommt wohl aus Berlin. Kannst du herausfinden, ob ihr Enkel in deinem Netzwerk ist?

★ zum Knobeln

1.4 Gruppierungen und Aggregatfunktionen

In SQL können Zeilen nach bestimmten Kriterien zusammengefasst werden.

Befehl	Bedeutung
`MIN(<Attribut>)` / `MAX(<Attribut>)`	kleinster bzw. größter Wert eines Attributs
`SUM(<Attribut>)` / `AVG(<Attribut>)`	Summe bzw. Durchschnitt der Werte eines Attributs
`COUNT(*)` / `COUNT(<Attribut>)`	zählt die Zeilen bzw. die Zeilen, in denen das Attribut nicht NULL ist
`GROUP BY <Attribut>`	fasst alle Zeilen mit gleichem Wert zu einer Gruppe zusammen; die Funktionen oben rechnen dann je Gruppe

GRÖSSE DES GRÖSSTEN USERS

```
SELECT MAX(centimeter) AS maximale_groesse
FROM users
```

LISTE DER STÄDTE UND DER JEWEILIGEN USER-ANZAHL

```
SELECT city, COUNT(*) AS anzahl
FROM users
GROUP BY city
ORDER BY anzahl DESC
```

Aufgabe 4

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Gib an, wann sich zuletzt ein Mitglied registriert hat.

b) Gib das Alter des ältesten Mitglieds an.

c) Betrachte die beiden Befehle und überlege, was diese ausgeben sollen. Teste deine Vermutung. Zu welchem Wert gehört der Name im zweiten Befehl? Erkläre, warum das Ergebnis anders ist als erwartet.

BEFEHL 1

```
SELECT name, MAX(centimeter)
FROM users
```

BEFEHL 2

```
SELECT name, MAX(centimeter),
       MIN(centimeter)
FROM users
```

d) Ermittle, wie viele männliche und wie viele weibliche User dein Netzwerk hat. Was steht außerdem in der Ausgabe?

e) ★ Prüfe, ob es User gibt, die am gleichen Tag Geburtstag haben. Wer ist es?

f) Gib die Namen der drei Städte mit den meisten Usern an.

g) Vergleiche die beiden Befehle. Warum kommen verschiedene Zahlen heraus? Finde die User, die im zweiten Befehl fehlen.

BEFEHL 1

```
SELECT COUNT(*)
FROM users
```

BEFEHL 2

```
SELECT COUNT(centimeter)
FROM users
```

h) Wie viele User kommen aus Hamburg? Vergleiche `city = 'Hamburg'` mit `city LIKE 'Hamburg%'`. Hast du wirklich alle Hamburger gefunden?

★ zum Knobeln